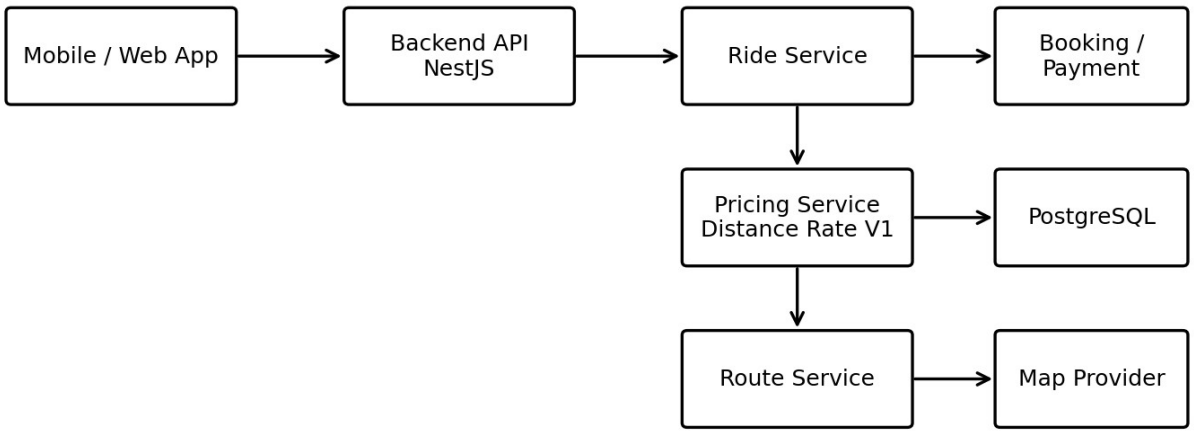
Baltic Carpooling App
V1 Ride Pricing Design

Developer Design Document - Node.js / TypeScript / PostgreSQL

Scope: No Fuel Price API in V1. Distance-based Poparide-style cost-sharing pricing.

V1 System Architecture - No Fuel Price API



Prepared for: Deliivo / Baltic inter-city carpooling MVP

1. Executive Summary

V1 pricing will use a simple distance-based cost-sharing model. The app will not fetch live fuel prices, will not calculate petrol/diesel/hybrid/electric differently, and will not ask drivers for fuel consumption. The backend will calculate route distance through a map provider, apply configurable EUR/km rates, and allow the driver to choose a price only within a safe cost-sharing range.

- Pricing version: DISTANCE_RATE_V1.
- Default region: BALTIC covering Estonia, Latvia, and Lithuania.
- Default recommended rate: EUR 0.08/km per passenger seat.
- Default allowed range: EUR 0.06/km to EUR 0.12/km.
- Default minimum seat price: EUR 3.
- Vehicle fuel type may be stored for profile completeness, but it is not used for V1 pricing.

2. Poparide Reference and Adaptation

The V1 model is inspired by public Poparide pricing principles, adapted for Baltic-market EUR pricing. Poparide publicly states that it caps prices at 20 cents/km to maintain fair pricing and that most routes average around 12-15 cents/km. Poparide also separates trip contribution from booking fees and payout fees, and positions its carpool product as cost sharing rather than commercial taxi service.

Poparide-style principle	Decision for Baltic V1
Distance-based pricing cap	Use configurable min/recommended/max EUR/km rates.
Fair cost-sharing language	Use terms like contribution, seat price, and cost-sharing amount.
Driver can set price but within rules	Driver can choose only inside backend-calculated min/max range.
Booking fee is separate from trip contribution	Pricing service calculates contribution only; payment service adds fees.
No taxi/profit positioning	Avoid fare, earning, taxi price, or profit wording in V1 UI.

3. V1 Product Decisions

Decision	V1 choice	Reason
Fuel price API	Not used	Adds complexity and external dependency without strong MVP value.
Fuel type pricing	Not used	Passengers understand city-to-city seat price better than fuel-consumption details.
Vehicle consumption	Not requested	Reduces ride-posting friction.
Route distance	Required from map provider	Core input for pricing.
Pricing config	Database driven	Allows admin tuning without code deployment.
Driver price edit	Allowed within min/max	Gives flexibility while preventing abuse.
Rounding	Nearest whole EUR	Simple and easy for passengers.

4. Pricing Formula

Main formula:

```
recommendedPricePerSeat = round(distanceKm * recommendedRatePerKm)
minAllowedPricePerSeat = round(distanceKm * minRatePerKm)
maxAllowedPricePerSeat = round(distanceKm * maxRatePerKm)
```

Minimum seat price protection:

```
recommendedPricePerSeat = max(minimumSeatPrice, recommendedPricePerSeat)
minAllowedPricePerSeat = max(minimumSeatPrice, minAllowedPricePerSeat)
```

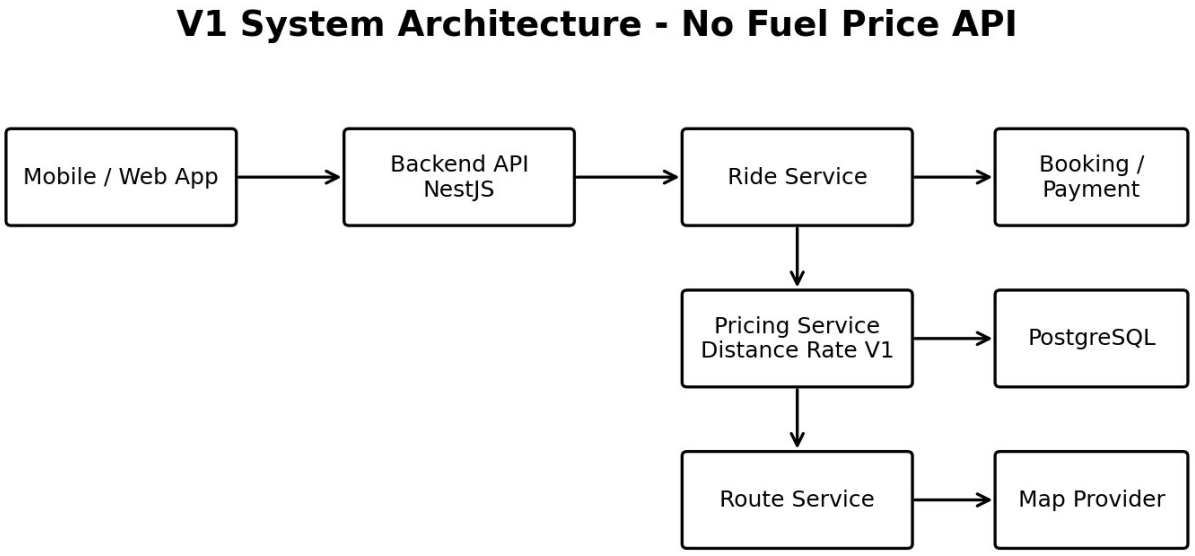
maxAllowedPricePerSeat = max(recommendedPricePerSeat, maxAllowedPricePerSeat)

Config field	Default value
region_code	BALTIC
currency	EUR
min_rate_per_km	0.0600
recommended_rate_per_km	0.0800
max_rate_per_km	0.1200
minimum_seat_price	3.00
rounding_strategy	NEAREST_EURO

4.1 Example Calculations

Route	Distance	Min	Recommended	Max
Tallinn -> Tartu	185 km	EUR 11	EUR 15	EUR 22
Tallinn -> Riga	310 km	EUR 19	EUR 25	EUR 37
Riga -> Vilnius	295 km	EUR 18	EUR 24	EUR 35
Tallinn -> Vilnius	600 km	EUR 36	EUR 48	EUR 72

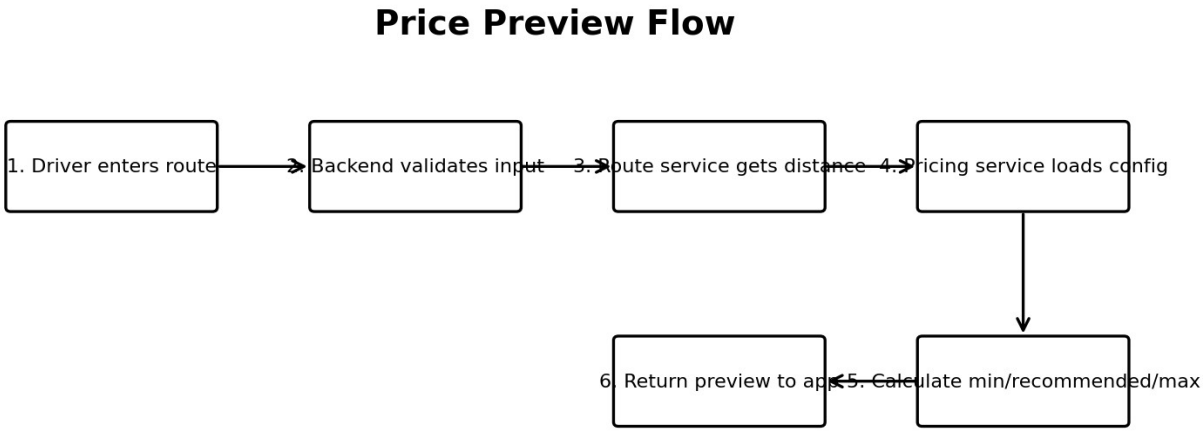
5. System Architecture



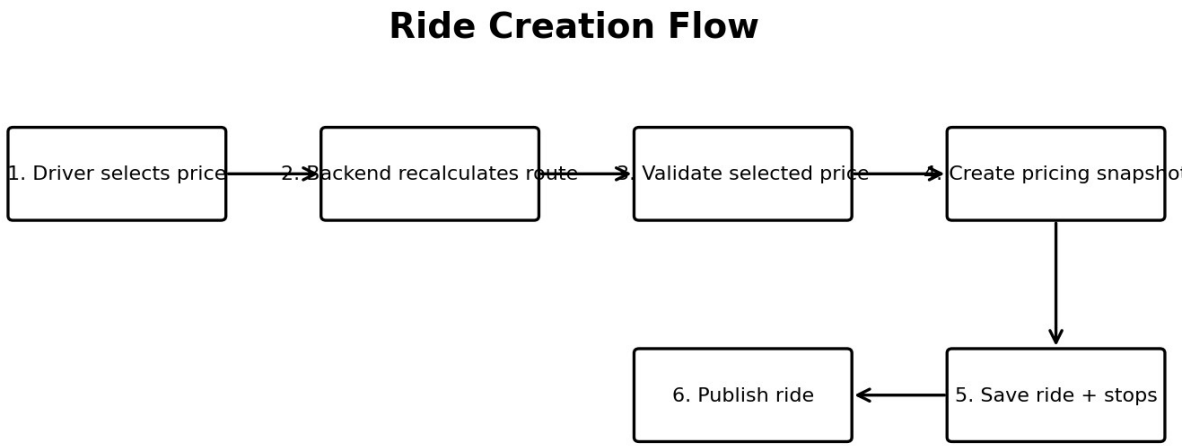
The pricing module should be isolated from ride, route, booking, and payment services. This makes it possible to replace DISTANCE_RATE_V1 with a future FUEL_BASED_V2 algorithm without changing unrelated modules.

6. Flow Diagrams

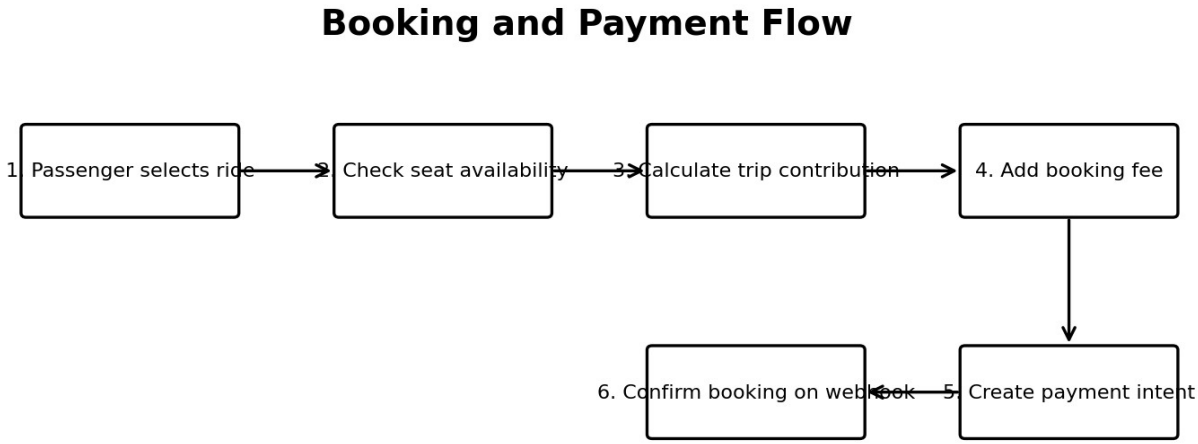
6.1 Price Preview Flow



6.2 Ride Creation Flow



6.3 Booking and Payment Flow



7. Node.js Implementation Architecture

- Recommended backend: NestJS + TypeScript.
- Database: PostgreSQL.
- ORM: Prisma or TypeORM. Prisma is recommended for fast MVP development.
- Map provider: Mapbox, Google Maps, HERE, or OpenRouteService. Choose one provider behind an interface.
- Payments: Stripe / Stripe Connect when payment and payout flow is implemented.

```
src/
  modules/
    rides/
    pricing/
    routes/
    bookings/
    payments/
    users/
    vehicles/
    notifications/
  common/
    errors/
    guards/
    pipes/
    utils/
```

8. API Contracts

8.1 Price Preview

POST /api/v1/rides/price-preview

Request:

```
{
  "origin": { "placeId": "place_tallinn", "name": "Tallinn, Estonia", "lat": 59.437, "lng": 24.7536, "countryCode": "EE" },
  "destination": { "placeId": "place_riga", "name": "Riga, Latvia", "lat": 56.9496, "lng": 24.1052, "countryCode": "LV" },
  "seatsOffered": 3
}
```

Response:

```
{
  "distanceKm": 310,
  "durationMinutes": 260,
  "currency": "EUR",
  "recommendedPricePerSeat": 25,
  "minAllowedPricePerSeat": 19,
  "maxAllowedPricePerSeat": 37,
  "minimumSeatPrice": 3,
  "pricingVersion": "DISTANCE_RATE_V1",
  "rateConfig": { "minRatePerKm": 0.06, "recommendedRatePerKm": 0.08, "maxRatePerKm": 0.12 }
}
```

8.2 Create Ride

POST /api/v1/rides

Request:

```
{
  "originPlaceId": "place_tallinn",
  "destinationPlaceId": "place_riga",
  "departureTime": "2026-07-10T09:00:00+03:00",
  "seatsOffered": 3,
  "selectedPricePerSeat": 25,
  "vehicleId": "veh_123",
  "description": "Small luggage only. Please be on time.",
  "stops": [{ "placeId": "place_parnu", "name": "Parnu", "sequence": 1 }]
}
```

Response:

```
{
  "rideId": "ride_123",
  "status": "PUBLISHED",
  "distanceKm": 310,
  "pricePerSeat": 25,
  "currency": "EUR",
  "availableSeats": 3,
  "pricing": {
    "recommendedPricePerSeat": 25,
    "minAllowedPricePerSeat": 19,
    "maxAllowedPricePerSeat": 37,
    "pricingVersion": "DISTANCE_RATE_V1"
  }
}
```

8.3 Validate Price

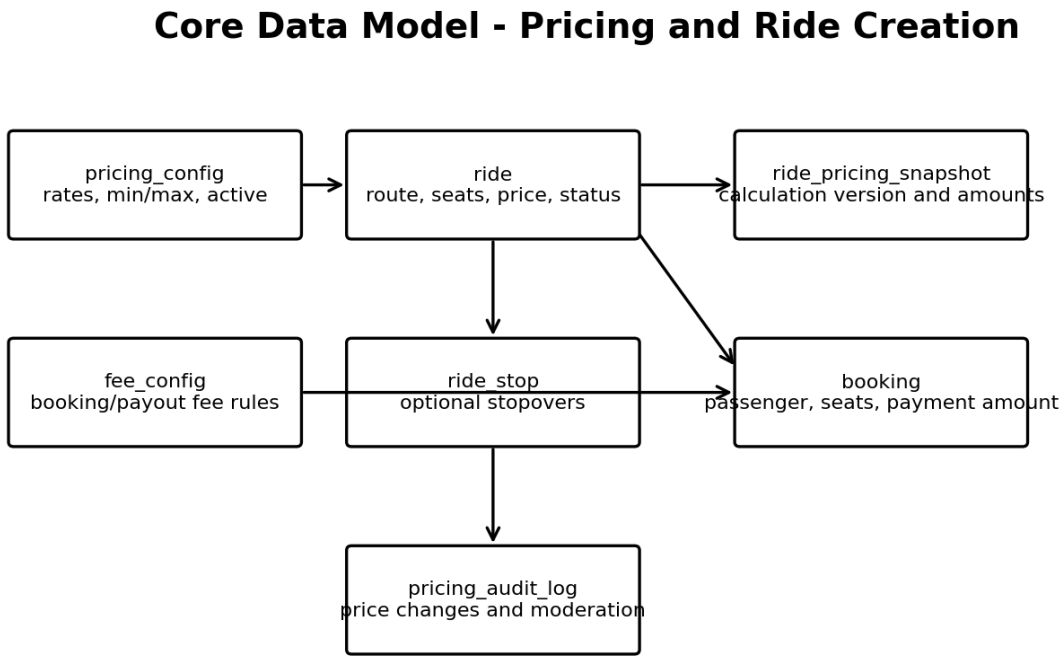
POST /api/v1/pricing/validate

Request:

```
{
  "distanceKm": 310,
  "selectedPricePerSeat": 40,
  "regionCode": "BALTIC"
```

}
Response:
{
  "valid": false,
  "reason": "PRICE_ABOVE_MAX_ALLOWED",
  "minAllowedPricePerSeat": 19,
  "maxAllowedPricePerSeat": 37,
  "message": "Please choose a price between EUR 19 and EUR 37."
}

9. Data Model



9.1 pricing_config

```
CREATE TABLE pricing_config (
  id UUID PRIMARY KEY,
  region_code VARCHAR(20) NOT NULL,
  currency VARCHAR(3) NOT NULL DEFAULT 'EUR',
  min_rate_per_km DECIMAL(8,4) NOT NULL,
  recommended_rate_per_km DECIMAL(8,4) NOT NULL,
  max_rate_per_km DECIMAL(8,4) NOT NULL,
  minimum_seat_price DECIMAL(8,2) NOT NULL DEFAULT 3.00,
  rounding_strategy VARCHAR(30) NOT NULL DEFAULT 'NEAREST_EURO',
  active BOOLEAN NOT NULL DEFAULT TRUE,
  valid_from TIMESTAMP NOT NULL DEFAULT now(),
  valid_to TIMESTAMP NULL,
  created_by UUID NULL,
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now()
);
```

9.2 ride

```
CREATE TABLE ride (
  id UUID PRIMARY KEY,
  driver_id UUID NOT NULL,
  vehicle_id UUID NULL,
  origin_place_id VARCHAR(255) NOT NULL,
  origin_name VARCHAR(255) NOT NULL,
  origin_lat DECIMAL(10,7) NOT NULL,
  origin_lng DECIMAL(10,7) NOT NULL,
  origin_country_code VARCHAR(2) NOT NULL,
  destination_place_id VARCHAR(255) NOT NULL,
  destination_name VARCHAR(255) NOT NULL,
  destination_lat DECIMAL(10,7) NOT NULL,
  destination_lng DECIMAL(10,7) NOT NULL,
  destination_country_code VARCHAR(2) NOT NULL,
  departure_time TIMESTAMP NOT NULL,
  timezone VARCHAR(64) NOT NULL,
  distance_km DECIMAL(8,2) NOT NULL,
  duration_minutes INT NULL,
  route_polyline TEXT NULL,
  seats_offered INT NOT NULL,
  seats_available INT NOT NULL,
  price_per_seat DECIMAL(8,2) NOT NULL,
  currency VARCHAR(3) NOT NULL DEFAULT 'EUR',
  status VARCHAR(30) NOT NULL DEFAULT 'PUBLISHED',
  description TEXT NULL,
  pricing_snapshot_id UUID NULL,
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now()
);
```

9.3 ride_pricing_snapshot

```
CREATE TABLE ride_pricing_snapshot (
  id UUID PRIMARY KEY,
  ride_id UUID NULL,
  pricing_version VARCHAR(50) NOT NULL,
  region_code VARCHAR(20) NOT NULL,
  currency VARCHAR(3) NOT NULL,
  distance_km DECIMAL(8,2) NOT NULL,
  min_rate_per_km DECIMAL(8,4) NOT NULL,
  recommended_rate_per_km DECIMAL(8,4) NOT NULL,
  max_rate_per_km DECIMAL(8,4) NOT NULL,
  minimum_seat_price DECIMAL(8,2) NOT NULL,
  recommended_price_per_seat DECIMAL(8,2) NOT NULL,
  min_allowed_price_per_seat DECIMAL(8,2) NOT NULL,
  max_allowed_price_per_seat DECIMAL(8,2) NOT NULL,
  selected_price_per_seat DECIMAL(8,2) NOT NULL,
  rounding_strategy VARCHAR(30) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT now()
);
```

9.4 booking

```
CREATE TABLE booking (
  id UUID PRIMARY KEY,
  ride_id UUID NOT NULL,
```

```
  passenger_id UUID NOT NULL,
  pickup_place_id VARCHAR(255) NOT NULL,
  pickup_name VARCHAR(255) NOT NULL,
  dropoff_place_id VARCHAR(255) NOT NULL,
  dropoff_name VARCHAR(255) NOT NULL,
  seats_booked INT NOT NULL DEFAULT 1,
  price_per_seat DECIMAL(8,2) NOT NULL,
  trip_contribution_amount DECIMAL(8,2) NOT NULL,
  booking_fee DECIMAL(8,2) NOT NULL DEFAULT 0,
  total_paid_amount DECIMAL(8,2) NOT NULL,
  currency VARCHAR(3) NOT NULL DEFAULT 'EUR',
  status VARCHAR(30) NOT NULL DEFAULT 'PENDING_PAYMENT',
  payment_intent_id VARCHAR(255) NULL,
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now()
);
```

10. TypeScript Pricing Implementation

```
export type RoundingStrategy = 'NEAREST_EURO' | 'NEAREST_HALF_EURO';

export interface PricingConfig {
  regionCode: string;
  currency: 'EUR';
  minRatePerKm: number;
  recommendedRatePerKm: number;
  maxRatePerKm: number;
  minimumSeatPrice: number;
  roundingStrategy: RoundingStrategy;
}

export interface PriceEstimateRequest {
  distanceKm: number;
  regionCode?: string;
  selectedPricePerSeat?: number;
}

export interface PriceEstimateResult {
  currency: 'EUR';
  distanceKm: number;
  recommendedPricePerSeat: number;
  minAllowedPricePerSeat: number;
  maxAllowedPricePerSeat: number;
  selectedPricePerSeat?: number;
  isSelectedPriceValid?: boolean;
  validationErrorCode?: string;
  pricingVersion: 'DISTANCE_RATE_V1';
}
export class DistanceRatePricingCalculator {
  calculate(request: PriceEstimateRequest, config: PricingConfig): PriceEstimateResult {
    if (request.distanceKm <= 0) throw new Error('INVALID_DISTANCE');

    const recommended = Math.max(
      config.minimumSeatPrice,
      this.round(request.distanceKm * config.recommendedRatePerKm, config.roundingStrategy),
```

```
    );

    const minAllowed = Math.max(
      config.minimumSeatPrice,
      this.round(request.distanceKm * config.minRatePerKm, config.roundingStrategy),
    );

    const maxAllowed = Math.max(
      recommended,
      this.round(request.distanceKm * config.maxRatePerKm, config.roundingStrategy),
    );

    const result: PriceEstimateResult = {
      currency: 'EUR',
      distanceKm: request.distanceKm,
      recommendedPricePerSeat: recommended,
      minAllowedPricePerSeat: minAllowed,
      maxAllowedPricePerSeat: maxAllowed,
      pricingVersion: 'DISTANCE_RATE_V1',
    };

    if (request.selectedPricePerSeat !== undefined) {
      const selected = request.selectedPricePerSeat;
      result.selectedPricePerSeat = selected;
      result.isSelectedPriceValid = selected >= minAllowed && selected <= maxAllowed;
      if (selected < minAllowed) result.validationErrorCode = 'PRICE_BELOW_MIN_ALLOWED';
      if (selected > maxAllowed) result.validationErrorCode = 'PRICE_ABOVE_MAX_ALLOWED';
    }

    return result;
  }

  private round(amount: number, strategy: RoundingStrategy): number {
    if (strategy === 'NEAREST_HALF_EURO') return Math.round(amount * 2) / 2;
    return Math.round(amount);
  }
}
```

11. Route Service Requirements

- The backend must calculate distance; the frontend must not be trusted for distance.
- Distance must be driving distance, not straight-line distance.
- If stops are provided, route should be origin -> stops in sequence -> destination.
- If the map provider fails, ride creation should fail with ROUTE_CALCULATION_FAILED.
- Polyline should be stored when available for display and later validation.

12. Stopover and Segment Pricing

For fastest V1, the app can start with direct city-to-city rides and one price per seat. If stopovers are included from day one, use segment pricing so passengers travelling only part of the route are charged fairly.

selectedRideRatePerKm = selectedPricePerSeat / fullRideDistanceKm
segmentPrice = max(minimumSeatPrice, round(segmentDistanceKm * selectedRideRatePerKm))

Passenger route	Distance	Example price

Tallinn -> Riga	310 km	EUR 25
Tallinn -> Parnu	130 km	EUR 10
Parnu -> Riga	180 km	EUR 15

13. Booking Fee and Payment Separation

Pricing service calculates only the trip contribution. Payment service adds passenger booking fee and manages payment collection, escrow/holding, refunds, and driver payout.

passengerTotal = tripContributionAmount + bookingFee
driverPayoutBase = tripContributionAmount
platformRevenue = bookingFee + optionalPayoutFee

Amount type	Owner service	Example
Trip contribution	Pricing / booking service	EUR 25
Passenger booking fee	Payment service	EUR 3.75 if 15%
Passenger total paid	Payment service	EUR 28.75
Driver payout basis	Payment service	EUR 25 before payout fee

14. Edge Cases and Expected Behavior

Edge case	Expected behavior
Map provider unavailable	Do not publish ride. Return ROUTE_CALCULATION_FAILED.
No active pricing config	Do not publish ride. Return ACTIVE_PRICING_CONFIG_NOT_FOUND.
Distance <= 0	Reject with INVALID_DISTANCE.
Very short ride	Apply minimumSeatPrice or reject below minimumRideDistanceKm.
Driver price below min	Reject with PRICE_BELOW_MIN_ALLOWED.
Driver price above max	Reject with PRICE_ABOVE_MAX_ALLOWED.
Pricing config changes after ride creation	Existing rides keep pricing snapshot. New rides use new config.
Driver edits route	Recalculate route and pricing; require confirmation.
Driver edits description only	No pricing recalculation needed.
Passenger books multiple seats	tripContributionAmount = pricePerSeat * seatsBooked.
Only one passenger books	Do not increase price. Listed price remains per seat.
Fuel type is electric/hybrid/diesel/petrol	Store in vehicle profile only; ignore for V1 pricing.
Cross-border ride	Use BALTIC region config in V1.
Existing booking and driver increases price	Do not increase price for existing passengers.

15. Validation and Anti-Abuse Rules

- Never trust frontend-provided distance, min price, max price, or recommended price.
- Backend must recalculate route and pricing during ride creation.
- Only one active pricing config should exist per region.
- Validate min_rate_per_km <= recommended_rate_per_km <= max_rate_per_km.
- Validate seatsOffered is within product limits, e.g. 1-7.
- Validate departure time is in the future.
- Audit price changes and validation failures for moderation.

16. Test Plan

Test area	Cases
Pricing calculator	310 km gives min 19, recommended 25, max 37 with default config.
Minimum price	20 km gives recommended 3 because minimumSeatPrice applies.

Validation	Selected price 25 valid; 10 too low; 50 too high.
Ride creation	Valid price publishes ride and creates pricing snapshot.
Map failure	Ride creation fails without saved ride.
Config missing	Ride creation fails cleanly.
Route edit	Distance and pricing snapshot recalculated.
Booking	Total contribution equals pricePerSeat * seatsBooked.
Stopover if enabled	Segment price proportional to segment distance and never below minimum.

17. Roadmap

Version	Scope
V1.0	Direct city-to-city rides, distance pricing, one BALTIC config, no fuel API.
V1.1	Stopovers and segment pricing, popular route hints, admin price tuning.
V2	Per-country pricing, demand/supply signals, optional fuel API, optional vehicle consumption.

18. Developer Checklist

- Create pricing_config table and seed BALTIC config.
- Create DistanceRatePricingCalculator.
- Create PricingService with estimate and validate methods.
- Create RouteService abstraction and one map provider implementation.
- Create price-preview endpoint.
- Integrate price validation in ride creation.
- Save ride_pricing_snapshot for every published ride.
- Keep booking fee logic out of PricingService.
- Add unit tests and integration tests.
- Add admin tools for pricing config management.

19. Sources Checked

Source	URL	Used for
Poparide Help - How to set a price for your trip	https://support.poparide.com/en/articles/9702565-how-to-set-a-price-for-your-trip	States maximum price per km and average price per km guidance.
Poparide Help - How to post a trip	https://support.poparide.com/en/articles/9647735-how-to-post-a-trip	Mentions fair pricing cap and similar trips.
Poparide Terms of Service	https://www.poparide.com/en-ca/terms	Separates trip contribution, booking fee, payout fee, and non-commercial driver obligations.
Poparide Help - Fees	https://support.poparide.com/en/articles/9704847-how-much-are-the-fees-to-use-poparide	Shows fee separation between passenger booking fee and driver payout fee.

End of document